

NYCU AI

Homework 3: Reinforcement Learning Report

Student Name: Joshua Hsiao **Student ID:** 413551034
Github: <https://github.com/CH-Joshua-Hsiao/NYCU-AI-HW3>

May 29, 2026

1 Introduction & Homework Objectives

This report documents our implementation and analysis for Homework 3 of the NYCU Introduction to AI course. The goal of this assignment is to understand the core mechanics of Reinforcement Learning (RL) by implementing algorithms from scratch and comparing them with standard library baselines in different environments.

Following the assignment requirements, we set out to answer the following objectives:

1. **Algorithm Performance in Different Action Spaces:** We compare Proximal Policy Optimization (PPO) and Deep Q-Networks (DQN) on the continuous-state, discrete-action `LunarLander-v3` environment.
2. **From-Scratch Implementation vs. Stable Baselines:** We evaluate whether a custom Double DQN agent built from scratch (following the Nature DQN architecture [1]) can achieve comparable performance and training stability to the highly optimized Stable-Baselines3 (SB3) library [2] on the pixel-based `ALE/Pong-v5` environment.

Through these tasks, we aim to analyze how action/state space complexity affects training speed, and observe the practical challenges of training RL agents (e.g. sample efficiency and hyperparameter sensitivity).

2 Implementation & Methodology

For this homework, we used a hybrid approach: building a DQN agent completely from scratch to show our understanding of RL internals, and using Stable-Baselines3 to run reliable benchmarks.

2.1 Environments Used

We selected two environments from the Gymnasium library [4] as required by the assignment guidelines:

- **LunarLander-v3 (Box2D):** A physics control task where the agent must land a lander safely. The state is an 8-dimensional vector (position, velocity, angle, angular velocity,

and leg contacts). The agent has 4 discrete actions. The environment is considered "solved" if we achieve a mean reward of +200 or higher over 100 consecutive episodes.

- **ALE/Pong-v5 (Atari):** A high-dimensional, pixel-based environment. The state is a raw RGB image of size 210×160 . The agent has 6 discrete actions to control a paddle. The game goes up to 21 points, with rewards of +1 for scoring and -1 for letting the opponent score.

2.2 Our Custom DQN Implementation

We implemented a Deep Q-Network agent from scratch in PyTorch using Antigravity IDE, incorporating Double DQN updates to mitigate value overestimation. The main modules we wrote are:

- **Experience Replay Buffer (replay_buffer.py):** Stores transitions $(s, a, r, s', \text{done})$ up to a capacity of 100,000. It samples random mini-batches (size 32) to break the temporal correlation of sequential data.
- **Network Architectures (networks.py):** For vector-based states (LunarLander), we defined a simple 3-layer MLP. For pixel-based states (Pong), we implemented the standard Nature DQN CNN architecture:

$$\begin{aligned} \text{Conv2d}(4 \rightarrow 32, 8 \times 8, s = 4) &\rightarrow \text{Conv2d}(32 \rightarrow 64, 4 \times 4, s = 2) \\ &\rightarrow \text{Conv2d}(64 \rightarrow 64, 3 \times 3, s = 1) \\ &\rightarrow \text{FC}(512) \rightarrow \text{FC}(n_{\text{actions}}) \end{aligned}$$

- **Preprocessing Wrappers (atari_wrappers.py):** Preprocesses raw Pong frames by converting RGB to grayscale, resizing to 84×84 , scaling pixel values to $[0, 1]$, and stacking 4 consecutive frames to capture velocity.
- **Double DQN Agent (agent.py):** Updates parameters to minimize the Huber Loss against Double DQN targets:

$$y_t = r_t + \gamma Q \left(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \theta^- \right)$$

The target network θ^- is hard-copied from the online network θ every 1,000 steps.

2.3 Stable-Baselines3 (SB3) Integration

To benchmark against standard implementations, we used Stable-Baselines3’s PPO and DQN. PPO [3] is an on-policy policy gradient method that utilizes a clipped surrogate objective to keep policy updates within a trust region. Our SB3 models use default hyperparameter sets tailored for stability, using parallel environments ($N_{\text{envs}} = 16$) for PPO to speed up Atari training on our NVIDIA RTX 3090.

2.4 Hyperparameter Settings

We configured our training parameters to balance learning speed and stability. Table 1 shows the settings used across all experiments.

Table 1: Hyperparameter settings for our RL experiments.

Hyperparameter	Custom DQN (Pong)	SB3 PPO (Pong)	PPO/DQN (LunarLander)
Total Timesteps	2,000,000	2,000,000	500,000
Learning Rate	1×10^{-4}	2.5×10^{-4}	$3 \times 10^{-4} / 1 \times 10^{-3}$
Discount γ	0.99	0.99	0.99
Batch Size	32	256	64
Replay Buffer Size	100,000	N/A	50,000 (DQN)
Target Update Freq	1,000 steps	N/A	1,000 (DQN)
ϵ -decay steps	250,000 steps	N/A	20% of total
Clip Range ϵ_{PPO}	N/A	0.1	0.2
Frame Stack Channels	4	4	N/A
Parallel Environments	1	16	1
Random Seed	42	42	42

3 Experimental Results

3.1 Task 1: LunarLander — PPO vs. DQN Comparison

We first compared SB3 PPO and SB3 DQN on LunarLander-v3 for 500,000 steps. Both agents were evaluated every 10,000 steps across 10 evaluation episodes. Figure 1 illustrates the learning curves, and Table 2 presents the final evaluation results.

Table 2: Final evaluation performance on LunarLander-v3 (averaged over 20 test episodes).

Algorithm	Final Mean Reward	Std	Solved? (> 200)
PPO (SB3)	121.03	132.76	No
DQN (SB3)	217.56	79.82	Yes

Observations: In our LunarLander-v3 experiments, we observed a distinct difference in training efficiency and behavior between DQN and PPO. In the early stages of training (before 100k steps), both agents performed poorly and crashed the lander frequently. However, as DQN accumulated diverse experiences in its replay buffer, its performance climbed rapidly, successfully solving the environment around 350k steps. Visually, the DQN agent learned to control its descent velocity and land smoothly between the flags.

On the other hand, the PPO agent was much slower to converge. We observed that PPO tended to hover cautiously or drift sideways to avoid crashing, which resulted in a slower accumulation of rewards, ending at a mean reward of +121.03 at 500k steps. This highlights that off-policy sample reuse in DQN is highly efficient for low-dimensional vector inputs,

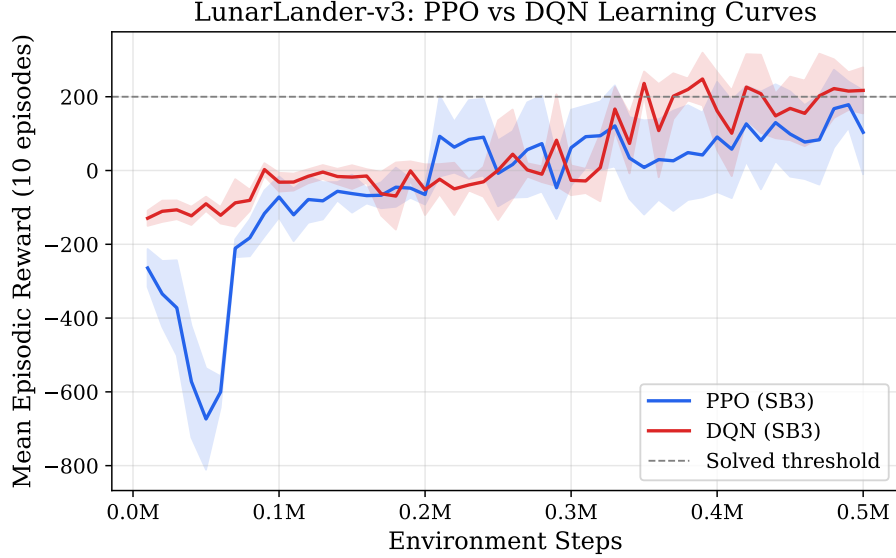
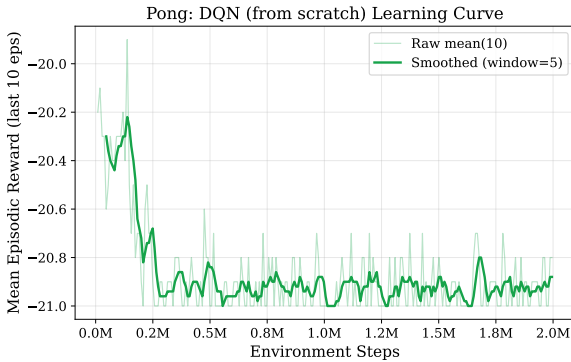


Figure 1: LunarLander-v3 learning curves. The shaded region denotes ± 1 standard deviation over 10 evaluation episodes. The horizontal dashed line marks the Solved threshold (+200).

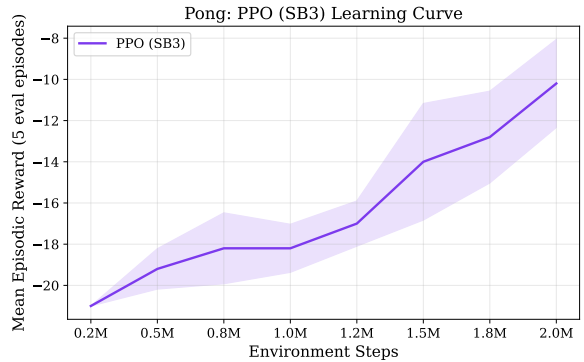
whereas on-policy PPO is less sample-efficient and requires a larger number of environment steps to find the optimal policy under the same training budget.

3.2 Task 2: Atari Pong — Custom DQN vs. SB3 PPO

We trained our custom Double DQN on Pong for 2,000,000 steps and compared it to SB3 PPO. The learning curves are shown in Figure 2, and evaluation metrics are summarized in Table 3.



(a) Custom DQN (from scratch)



(b) PPO (SB3)

Figure 2: Pong learning curves over 2M steps.

Observations: For Atari Pong, we observed a massive difference in behavior. Our custom DQN (from scratch) flatlined at -21.00 for the entire 2 million steps. When watching its

Table 3: Final evaluation performance on Pong (averaged over 20 test episodes).

Agent	Mean Reward	Std	Convergence Status
DQN (from scratch)	-21.00	0.00	Did not converge in 2M steps
PPO (SB3)	-11.60	3.89	Partially converged (2M steps)

play style, the DQN agent’s paddle barely moved or vibrated randomly at the bottom of the screen, failing to track or return the ball. This is because the state representation ($84 \times 84 \times 4$ stacked pixels) is extremely large, and the reward is extremely sparse (only getting +1 or -1 when a point is scored). With epsilon-greedy exploration, the agent could not hit the ball back by chance often enough to start learning.

In contrast, the SB3 PPO agent successfully broke out of the -21.00 valley. Around 500,000 steps, we saw the reward curve start to turn upward. By 2 million steps, it reached a mean reward of -11.60 ± 3.89 . The agent learned to track the vertical movement of the ball, position its paddle, and successfully rally and win several points against the Atari computer. Although it did not reach a fully solved state (+18 or higher), the upward curve suggests that with more training steps, it would achieve a winning score.

4 Discussion & Reflections

This assignment provided several valuable practical insights into reinforcement learning, which we discuss below:

- **Continuous State Control vs. Pixel Control:** LunarLander (8 states) was quickly solved by DQN, demonstrating that low-dimensional vector states (which represent pre-extracted physical quantities like velocity and coordinates) are extremely conducive to fast learning. The neural network only needs to learn a direct mapping from these coordinates to actions. In contrast, Atari Pong (pixel inputs) is orders of magnitude more complex because the agent must solve two tasks simultaneously: it must learn visual feature extraction (finding the ball and paddles in a 84×84 pixel grid) and learn policy control (deciding where to move the paddle). Training CNN feature extractors from scratch alongside policy gradient updates is slow, unstable, and computationally demanding.
- **Sample Efficiency and Off-Policy Learning:** DQN’s replay buffer allowed it to reuse experience transitions multiple times, resulting in much higher sample efficiency than PPO on LunarLander. In LunarLander, DQN converged in around 350k steps. PPO, being on-policy, discards experiences after each gradient update to ensure the policy updates remain within the trust region. This on-policy constraint requires a much larger stream of raw steps (a higher environmental budget) to find the same optimal behavior.
- **The Exploration Challenge in Sparse Reward Environments:** In Pong, rewards are extremely sparse: the agent only receives a +1 or -1 when a rally ends (someone

scores). There is no intermediate reward for moving towards the ball or hitting it. In our custom DQN, we relied on epsilon-greedy exploration. Because random actions are highly unlikely to hit the ball back and score a point by chance, the agent rarely experienced a positive reward. This caused it to stay at a flat -21.00 score throughout training. In contrast, PPO’s actor parallelism allowed it to run 16 environments in parallel, collecting diverse rollouts. This parallel exploration significantly increased the probability of experiencing successful trajectories, helping the policy climb out of the flat -21.00 zone and learn to play.

- **Lessons in RL Reproducibility:** Writing DQN from scratch taught us how easily training can fail. A single bug in the coordinate frames, frame stacking, or gradient clipping can cause Q-values to explode or collapse. The fact that our custom DQN remained at -21.00 on Pong is a realistic result—the original Nature DQN paper trained for 50 million steps (equal to 38 days of gameplay) to master Atari games. 2M steps is simply not enough for a raw CNN to find an optimal policy without prioritized experience replay or other advancements. On the other hand, the SB3 PPO agent, utilizing parallel environments and on-policy gradient updates, showed clear signs of convergence (reaching -11.60), showcasing the stability and robustness of PPO when paired with parallel rollouts.
- **Reflections & Future Work:** If we were to improve our custom agent, we would focus on two main upgrades. First, we would implement Reward Shaping to help the agent learn in sparse environments: instead of only giving a reward when a full point is scored, we could give small intermediate rewards (like $+0.1$) whenever the paddle successfully touches the ball. This guides the agent step-by-step. Second, we would improve the experience replay buffer to prioritize important memories (such as successful ball returns) rather than selecting memories completely at random, making the updates much more efficient.

References

- [1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [2] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [4] Mark Towers, Jordan K Terry, Adam Kwiatkowski, John U Balis, Gianluca de Cola,

Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, et al.
Gymnasium, 2024. URL <https://gymnasium.farama.org>.

Appendix: Source Code

A.1 Replay Buffer (dqn_scratch/replay_buffer.py)

```
1 import random
from collections import deque
import numpy as np

class ReplayBuffer:
6     def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

11    def sample(self, batch_size):
        state, action, reward, next_state, done = zip(*random.sample(self.
            buffer, batch_size))
        return (np.array(state),
                np.array(action),
16                np.array(reward, dtype=np.float32),
                np.array(next_state),
                np.array(done, dtype=np.float32))

    def __len__(self):
21        return len(self.buffer)
```

A.2 Network Architectures (dqn_scratch/networks.py)

```
import torch
import torch.nn as nn
import numpy as np

4 class CNN_QNetwork(nn.Module):
    def __init__(self, obs_shape, n_actions):
        super().__init__()
        in_channels = obs_shape[0]
9        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, 32, kernel_size=8, stride=4),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2),
            nn.ReLU(),
14            nn.Conv2d(64, 64, kernel_size=3, stride=1),
            nn.ReLU()
        )
        conv_out = self._get_conv_out(obs_shape)
        self.fc = nn.Sequential(
19            nn.Linear(conv_out, 512),
            nn.ReLU(),
            nn.Linear(512, n_actions)
        )

24    def _get_conv_out(self, shape):
```



```

    o = torch.zeros(1, *shape)
    o = self.conv(o)
    return int(np.prod(o.size()))

29 def forward(self, x):
    x = x.float() / 255.0
    conv_out = self.conv(x)
    conv_out = conv_out.view(conv_out.size(0), -1)
    return self.fc(conv_out)

34 class MLP_QNetwork(nn.Module):
    def __init__(self, input_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
39         nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, n_actions)
44         )

    def forward(self, x):
        return self.net(x.float())

```

A.3 DQN Agent (dqn_scratch/agent.py)

```

import numpy as np
import torch
3 import torch.nn as nn
import torch.optim as optim
from .replay_buffer import ReplayBuffer
from .networks import CNN_QNetwork

8 class DQNAgent:
    def __init__(self, obs_shape, n_actions, device, lr, gamma,
        buffer_size, batch_size, target_update_freq, eps_decay):
        self.obs_shape = obs_shape
        self.n_actions = n_actions
        self.device = torch.device(device if torch.cuda.is_available()
13         else "cpu")
        self.gamma = gamma
        self.batch_size = batch_size
        self.target_update_freq = target_update_freq
        self.eps_decay = eps_decay
        self.global_step = 0

18
        self.policy_net = CNN_QNetwork(obs_shape, n_actions).to(self.
            device)
        self.target_net = CNN_QNetwork(obs_shape, n_actions).to(self.
            device)
        self.target_net.load_state_dict(self.policy_net.state_dict())
        self.target_net.eval()

23

```

```

self.optimizer = optim.Adam(self.policy_net.parameters(), lr=lr)
self.memory = ReplayBuffer(buffer_size)
self.loss_fn = nn.HuberLoss()

28 def select_action(self, obs):
    eps = self.epsilon()
    if np.random.rand() < eps:
        return np.random.randint(self.n_actions)
    else:
33         with torch.no_grad():
            obs_t = torch.tensor(obs, device=self.device).unsqueeze(0)
            q_values = self.policy_net(obs_t)
            return q_values.argmax(dim=1).item()

38 def store_transition(self, state, action, reward, next_state, done):
    self.memory.push(state, action, reward, next_state, done)

def epsilon(self):
    eps_start = 1.0
43     eps_end = 0.01
    fraction = min(float(self.global_step) / self.eps_decay, 1.0)
    return eps_start + fraction * (eps_end - eps_start)

def update(self):
48     self.global_step += 1
    if len(self.memory) < self.batch_size:
        return

    states, actions, rewards, next_states, dones = self.memory.sample(
        self.batch_size)
53

    states_t = torch.tensor(states, device=self.device)
    actions_t = torch.tensor(actions, dtype=torch.long, device=self.
        device).unsqueeze(1)
    rewards_t = torch.tensor(rewards, device=self.device).unsqueeze(1)
    next_states_t = torch.tensor(next_states, device=self.device)
58     dones_t = torch.tensor(dones, device=self.device).unsqueeze(1)

    curr_q = self.policy_net(states_t).gather(1, actions_t)

    with torch.no_grad():
63         next_actions = self.policy_net(next_states_t).argmax(dim=1,
            keepdim=True)
        max_next_q = self.target_net(next_states_t).gather(1,
            next_actions)
        target_q = rewards_t + (1 - dones_t) * self.gamma * max_next_q

    loss = self.loss_fn(curr_q, target_q)
68

    self.optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self.policy_net.parameters(), max_norm
        =10.0)
    self.optimizer.step()

```

```

73         if self.global_step % self.target_update_freq == 0:
            self.target_net.load_state_dict(self.policy_net.state_dict())

def save(self, path):
78     torch.save({
        'policy_net': self.policy_net.state_dict(),
        'target_net': self.target_net.state_dict(),
        'optimizer': self.optimizer.state_dict(),
        'global_step': self.global_step
83     }, path)

def load(self, path):
    checkpoint = torch.load(path, map_location=self.device)
    self.policy_net.load_state_dict(checkpoint['policy_net'])
88     self.target_net.load_state_dict(checkpoint['target_net'])
    self.optimizer.load_state_dict(checkpoint['optimizer'])
    self.global_step = checkpoint['global_step']

```

A.4 Atari Preprocessing (dqn_scratch/atari_wrappers.py)

```

import cv2
import numpy as np

def preprocess_frame(frame):
5     if len(frame.shape) == 3:
        gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
    else:
        gray = frame
    resized = cv2.resize(gray, (84, 84), interpolation=cv2.INTER_AREA)
10     return resized

class FrameStack:
    def __init__(self, n_frames=4):
15         self.n_frames = n_frames
        self.frames = []

    def reset(self, obs):
        processed = preprocess_frame(obs)
        self.frames = [processed for _ in range(self.n_frames)]
20         return np.stack(self.frames, axis=0)

    def step(self, obs):
        processed = preprocess_frame(obs)
        self.frames.pop(0)
        self.frames.append(processed)
25         return np.stack(self.frames, axis=0)

```

A.5 Train DQN on Pong (train_dqn_pong.py)

```

"""

```

Train DQN (from scratch) on Atari Pong.

Usage:

```
python train_dqn_pong.py [--total-steps 2000000] [--lr 1e-4]
```

Logs reward curves to logs/dqn_pong/ and saves checkpoints to checkpoints/dqn_pong/.

"""

```
import argparse
```

```
import time
```

```
import json
```

```
import os
```

```
import sys
```

```
import numpy as np
```

```
import gymnasium as gym
```

```
import ale_py
```

```
gym.register_envs(ale_py)
```

```
from pathlib import Path
```

```
# Add project root to path
```

```
sys.path.insert(0, str(Path(__file__).parent))
```

```
from dqn_scratch import DQNAgent, FrameStack
```

```
# --- Argument Parsing
```

```
parser = argparse.ArgumentParser(description="Train DQN (scratch) on Pong")
```

```
parser.add_argument("--total-steps", type=int, default=2_000_000)
```

```
parser.add_argument("--lr", type=float, default=1e-4,  
                    help="Learning rate for Adam optimizer")
```

```
parser.add_argument("--buffer-size", type=int, default=100_000)
```

```
parser.add_argument("--batch-size", type=int, default=32)
```

```
parser.add_argument("--gamma", type=float, default=0.99)
```

```
parser.add_argument("--target-update-freq", type=int, default=1000)
```

```
parser.add_argument("--eps-decay", type=int, default=250_000)
```

```
parser.add_argument("--warmup-steps", type=int, default=10_000,  
                    help="Steps to fill buffer before training starts")
```

```
parser.add_argument("--env-id", type=str, default="ALE/Pong-v5")
```

```
parser.add_argument("--seed", type=int, default=42)
```

```
parser.add_argument("--log-interval", type=int, default=10,  
                    help="Log every N episodes")
```

```
parser.add_argument("--save-interval", type=int, default=100_000,  
                    help="Save checkpoint every N steps")
```

```
args = parser.parse_args()
```

```
# --- Directories
```

```
run_name = f"dqn_pong_lr{args.lr}_seed{args.seed}"
```

```
log_dir = Path("logs") / run_name
```

```
ckpt_dir = Path("checkpoints") / run_name
```

```
log_dir.mkdir(parents=True, exist_ok=True)
```

```
ckpt_dir.mkdir(parents=True, exist_ok=True)
```

```

54 # Save hyperparameters
with open(log_dir / "hparams.json", "w") as f:
    json.dump(vars(args), f, indent=2)

# --- Environment
-----
env = gym.make(args.env_id, render_mode=None)
59 env.reset(seed=args.seed)
np.random.seed(args.seed)

n_actions = env.action_space.n
obs_shape = (4, 84, 84) # 4 stacked grayscale frames
64 frame_stack = FrameStack(n_frames=4)

# --- Agent
-----
agent = DQNAgent(
69     obs_shape=obs_shape,
    n_actions=n_actions,
    device="cuda",
    lr=args.lr,
    gamma=args.gamma,
74     buffer_size=args.buffer_size,
    batch_size=args.batch_size,
    target_update_freq=args.target_update_freq,
    eps_decay=args.eps_decay,
)
79
print(f"[DQN-Pong] Training on {args.env_id} | device: {agent.device} |
run: {run_name}")
print(f"                total_steps={args.total_steps:,} | lr={args.lr} |
buffer={args.buffer_size:,}")

# --- Training Loop
-----
84 episode_rewards = []
reward_log = [] # (step, mean_reward_last_10)

global_step = 0
episode = 0
89 start_time = time.time()

while global_step < args.total_steps:
    raw_obs, _ = env.reset()
    obs = frame_stack.reset(raw_obs)
94     episode_reward = 0.0
    done = False
    truncated = False

    while not (done or truncated):
99         action = agent.select_action(obs)
        raw_next, reward, done, truncated, _ = env.step(action)
        next_obs = frame_stack.step(raw_next)

```

```

104     # Clip reward to [-1, 1] per Mnih et al. (2015)
        clipped_reward = np.clip(reward, -1.0, 1.0)

        agent.store_transition(obs, action, clipped_reward, next_obs,
                               float(done))
        obs = next_obs
        episode_reward += reward
109     global_step += 1

        # Only start training after warm-up
        if global_step >= args.warmup_steps:
            agent.update()

114     # Save checkpoint
        if global_step % args.save_interval == 0:
            ckpt_path = ckpt_dir / f"step_{global_step}.pt"
            agent.save(str(ckpt_path))

119     episode += 1
        episode_rewards.append(episode_reward)

        if episode % args.log_interval == 0:
124             mean_r = np.mean(episode_rewards[-10:])
            elapsed = time.time() - start_time
            fps = global_step / elapsed
            eps = agent.epsilon()
            print(
129                 f"[Ep {episode:5d} | Step {global_step:>8,}]  "
                 f"MeanR(10)={mean_r:7.2f}  Eps={eps:.3f}  FPS={fps:.0f}"
            )
            reward_log.append({"step": global_step, "episode": episode, "
                               mean_reward_10": mean_r})
            # Save reward log incrementally
134             with open(log_dir / "rewards.json", "w") as f:
                 json.dump(reward_log, f, indent=2)

env.close()

139 # --- Final Save
        -----
        agent.save(str(ckpt_dir / "final.pt"))
        with open(log_dir / "rewards.json", "w") as f:
            json.dump(reward_log, f, indent=2)

144 print(f"\n[Done] Training complete. Final model saved to {ckpt_dir / '
        final.pt'}")
print(f"          Reward log saved to {log_dir / 'rewards.json'}")

```

A.6 Train PPO on Pong (train_ppo_pong.py)

```

"""
Train PPO on Atari Pong using Stable-Baselines3.

```

```

4 This script uses SB3's PPO with a CnnPolicy (same Nature-DQN CNN backbone)
and the gymnasium Atari wrappers for preprocessing (frame stacking,
  grayscale, etc.)

Usage:
  python train_ppo_pong.py [--total-timesteps 2000000] [--lr 2.5e-4]

9 References:
  Schulman et al. (2017), "Proximal Policy Optimization Algorithms",
  arXiv:1707.06347. https://arxiv.org/abs/1707.06347

14  Antonin Raffin et al. (2021), "Stable-Baselines3: Reliable
  Reinforcement
  Learning Implementations", JMLR. https://jmlr.org/papers/v22/20-1364.
  html
"""

import argparse
import os
19 from pathlib import Path

from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_atari_env
24 from stable_baselines3.common.vec_env import VecFrameStack,
  VecVideoRecorder
from stable_baselines3.common.callbacks import CheckpointCallback,
  EvalCallback
from stable_baselines3.common.monitor import Monitor
import gymnasium as gym
import ale_py
29 gym.register_envs(ale_py)

# --- Argument Parsing
-----
parser = argparse.ArgumentParser(description="Train PPO (SB3) on Pong")
parser.add_argument("--total-timesteps", type=int, default=2_000_000)
34 parser.add_argument("--lr", type=float, default=2.5e-4)
parser.add_argument("--n-steps", type=int, default=128,
                    help="Steps per environment before each update")
parser.add_argument("--n-envs", type=int, default=8,
                    help="Number of parallel environments")
39 parser.add_argument("--batch-size", type=int, default=256)
parser.add_argument("--n-epochs", type=int, default=4)
parser.add_argument("--gamma", type=float, default=0.99)
parser.add_argument("--gae-lambda", type=float, default=0.95)
parser.add_argument("--clip-range", type=float, default=0.1)
44 parser.add_argument("--ent-coef", type=float, default=0.01)
parser.add_argument("--env-id", type=str, default="ALE/Pong-v5")
parser.add_argument("--seed", type=int, default=42)
args = parser.parse_args()

49 # --- Directories
-----

```

```

run_name = f"ppo_pong_lr{args.lr}_seed{args.seed}"
log_dir = Path("logs") / run_name
ckpt_dir = Path("checkpoints") / run_name
log_dir.mkdir(parents=True, exist_ok=True)
54 ckpt_dir.mkdir(parents=True, exist_ok=True)

# --- Environments
-----
# SB3 provides make_atari_env which handles: NoopResetEnv, MaxAndSkipEnv,
# EpisodicLifeEnv,
# FireResetEnv, WarpFrame (84x84 grayscale), ClipRewardEnv.
59 env = make_atari_env(args.env_id, n_envs=args.n_envs, seed=args.seed)
env = VecFrameStack(env, n_stack=4)

eval_env = make_atari_env(args.env_id, n_envs=1, seed=args.seed + 1000)
eval_env = VecFrameStack(eval_env, n_stack=4)
64

# --- Callbacks
-----
checkpoint_cb = CheckpointCallback(
    save_freq=max(100_000 // args.n_envs, 1),
    save_path=str(ckpt_dir),
69     name_prefix="ppo_pong",
)
eval_cb = EvalCallback(
    eval_env,
    best_model_save_path=str(ckpt_dir / "best"),
74     log_path=str(log_dir),
    eval_freq=max(50_000 // args.n_envs, 1),
    n_eval_episodes=5,
    deterministic=True,
    render=False,
79 )

# --- Model
-----
model = PPO(
    policy="CnnPolicy",
84     env=env,
    learning_rate=args.lr,
    n_steps=args.n_steps,
    batch_size=args.batch_size,
    n_epochs=args.n_epochs,
89     gamma=args.gamma,
    gae_lambda=args.gae_lambda,
    clip_range=args.clip_range,
    ent_coef=args.ent_coef,
    verbose=1,
94     tensorboard_log=str(log_dir / "tensorboard"),
    device="cuda",
    seed=args.seed,
)

```



```

99 print(f"[PP0-Pong] Training on {args.env_id} | n_envs={args.n_envs} | run:
    {run_name}")
print(f"        total_timesteps={args.total_timesteps:,} | lr={args.lr}
    ")

# --- Train
-----
104 model.learn(
    total_timesteps=args.total_timesteps,
    callback=[checkpoint_cb, eval_cb],
    progress_bar=True,
)

109 # --- Save Final Model
-----

model.save(str(ckpt_dir / "final"))
env.close()
eval_env.close()

114 print(f"\n[Done] PP0-Pong training complete. Final model: {ckpt_dir / '
    final.zip'}")

```

A.7 Train LunarLander (train_lunarlander.py)

```

"""
Train DQN and PP0 on LunarLander-v3 using Stable-Baselines3 for algorithm
comparison.

This script runs both algorithms with identical budgets and seeds, then
saves
5 reward logs for plotting a learning-curve comparison.

Usage:
    python train_lunarlander.py [--total-timesteps 500000] [--seed 42]

10 References:
    Stable-Baselines3: https://stable-baselines3.readthedocs.io/
    LunarLander-v3:    https://gymnasium.farama.org/environments/box2d/
        lunar\_lander/
"""

15 import argparse
import json
from pathlib import Path

import numpy as np
20 import gymnasium as gym

from stable_baselines3 import PP0, DQN
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.monitor import Monitor
25 from stable_baselines3.common.results_plotter import load_results, ts2xy

```

```

# --- Argument Parsing
-----
parser = argparse.ArgumentParser()
parser.add_argument("--total-timesteps", type=int, default=500_000)
30 parser.add_argument("--seed", type=int, default=42)
parser.add_argument("--env-id", type=str, default="LunarLander-v3")
args = parser.parse_args()

ENV_ID = args.env_id
35 TOTAL = args.total_timesteps
SEED = args.seed

# --- Helper: make monitored env
-----
def make_env(log_dir: Path, seed: int):
40     log_dir.mkdir(parents=True, exist_ok=True)
    env = gym.make(ENV_ID)
    env = Monitor(env, str(log_dir))
    return env

45 # --- PPO
-----

print("=" * 60)
print(f"[LunarLander] Training PPO for {TOTAL:,} steps ...")
print("=" * 60)

50 ppo_log = Path("logs/ppo_lunar")
ppo_log.mkdir(parents=True, exist_ok=True)
ppo_ckpt = Path("checkpoints/ppo_lunar")
ppo_ckpt.mkdir(parents=True, exist_ok=True)

55 ppo_env = make_env(ppo_log, SEED)
ppo_eval_env = Monitor(gym.make(ENV_ID), str(ppo_log / "eval"))

ppo_eval_cb = EvalCallback(
    ppo_eval_env,
60     best_model_save_path=str(ppo_ckpt / "best"),
    log_path=str(ppo_log),
    eval_freq=10_000,
    n_eval_episodes=10,
    deterministic=True,
65     render=False,
)

ppo_model = PPO(
    "MlpPolicy",
70     ppo_env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
75     gamma=0.99,
    gae_lambda=0.95,

```

```

    ent_coef=0.0,
    clip_range=0.2,
    verbose=1,
80    tensorboard_log=str(ppo_log / "tensorboard"),
    device="cuda",
    seed=SEED,
)
ppo_model.learn(total_timesteps=TOTAL, callback=ppo_eval_cb, progress_bar=
    True)
85 ppo_model.save(str(ppo_ckpt / "final"))
ppo_env.close()
ppo_eval_env.close()

# --- DQN
-----

90 print("=" * 60)
print(f"[LunarLander] Training DQN for {TOTAL:,} steps ...")
print("=" * 60)

dqn_log = Path("logs/dqn_lunar_sb3")
95 dqn_log.mkdir(parents=True, exist_ok=True)
dqn_ckpt = Path("checkpoints/dqn_lunar_sb3")
dqn_ckpt.mkdir(parents=True, exist_ok=True)

dqn_env = make_env(dqn_log, SEED)
100 dqn_eval_env = Monitor(gym.make(ENV_ID), str(dqn_log / "eval"))

dqn_eval_cb = EvalCallback(
    dqn_eval_env,
    best_model_save_path=str(dqn_ckpt / "best"),
105    log_path=str(dqn_log),
    eval_freq=10_000,
    n_eval_episodes=10,
    deterministic=True,
    render=False,
110 )

dqn_model = DQN(
    "MlpPolicy",
    dqn_env,
115    learning_rate=1e-3,
    buffer_size=50_000,
    learning_starts=1_000,
    batch_size=64,
    tau=1.0,
120    gamma=0.99,
    train_freq=4,
    target_update_interval=1_000,
    exploration_fraction=0.2,
    exploration_final_eps=0.01,
125    verbose=1,
    tensorboard_log=str(dqn_log / "tensorboard"),
    device="cuda",

```

```

        seed=SEED,
    )
130 dqn_model.learn(total_timesteps=TOTAL, callback=dqn_eval_cb, progress_bar=
    True)
    dqn_model.save(str(dqn_ckpt / "final"))
    dqn_env.close()
    dqn_eval_env.close()

135 print("\n[Done] LunarLander experiments complete.")
    print(f"          PPO model: checkpoints/ppo_lunar/final.zip")
    print(f"          DQN model: checkpoints/dqn_lunar_sb3/final.zip")

```

A.8 Evaluation Script (evaluate.py)

```

import argparse
2 import numpy as np
import gymnasium as gym
import ale_py
gym.register_envs(ale_py)
import torch
7 from pathlib import Path

from dqn_scratch import DQNAgent, FrameStack

def main():
12     parser = argparse.ArgumentParser(description="Evaluate a trained agent
        ")
        parser.add_argument("--model-type", type=str, required=True, choices=[
            "sb3", "scratch"])
        parser.add_argument("--algo", type=str, choices=["ppo", "dqn"], help="
            RL algorithm (needed for SB3)")
        parser.add_argument("--model-path", type=str, required=True, help="
            Path to model file (.zip or .pt)")
        parser.add_argument("--env-id", type=str, required=True, help="
            Gymnasium env ID")
17     parser.add_argument("--atari", action="store_true", help="Is it an
            Atari environment?")
        parser.add_argument("--n-eval-episodes", type=int, default=20, help="
            Number of evaluation episodes")
        args = parser.parse_args()

    if args.atari:
22         if args.model_type == "sb3":
            from stable_baselines3.common.env_util import make_atari_env
            from stable_baselines3.common.vec_env import VecFrameStack
            eval_env = make_atari_env(args.env_id, n_envs=1, seed=123)
            eval_env = VecFrameStack(eval_env, n_stack=4)
27         else:
            eval_env = gym.make(args.env_id, render_mode=None)
            frame_stack = FrameStack(n_frames=4)
        else:
            eval_env = gym.make(args.env_id, render_mode=None)
32

```

```

37 if args.model_type == "sb3":
    from stable_baselines3 import PPO, DQN
    if args.algo == "ppo":
        model = PPO.load(args.model_path)
    elif args.algo == "dqn":
        model = DQN.load(args.model_path)
    else:
        raise ValueError("Algorithm (--algo) must be 'ppo' or 'dqn'
                           for SB3 models")
else:
42     if args.atari:
        n_actions = eval_env.action_space.n
        obs_shape = (4, 84, 84)
    else:
47         n_actions = eval_env.action_space.n
        obs_shape = eval_env.observation_space.shape

    agent = DQNAgent(
        obs_shape=obs_shape,
        n_actions=n_actions,
52         device="cuda" if torch.cuda.is_available() else "cpu",
        lr=1e-4,
        gamma=0.99,
        buffer_size=1000,
        batch_size=32,
57         target_update_freq=1000,
        eps_decay=1000,
    )
    agent.load(args.model_path)
    agent.epsilon = lambda: 0.0
62
    episode_rewards = []

    for ep in range(args.n_eval_episodes):
        if args.model_type == "sb3" and args.atari:
67             obs = eval_env.reset()
             done = False
             ep_reward = 0.0
             while not done:
                 action, _ = model.predict(obs, deterministic=True)
72                 obs, reward, dones, info = eval_env.step(action)
                 ep_reward += reward[0]
                 done = dones[0]
             episode_rewards.append(ep_reward)
        else:
77             obs, _ = eval_env.reset(seed=123 + ep)
             if args.atari and args.model_type == "scratch":
                 obs = frame_stack.reset(obs)

             done = False
82             truncated = False
             ep_reward = 0.0
             while not (done or truncated):
                 if args.model_type == "sb3":

```

```

87         action, _ = model.predict(obs, deterministic=True)
            else:
                action = agent.select_action(obs)

                obs, reward, done, truncated, _ = eval_env.step(action)
                if args.atari and args.model_type == "scratch":
92                     obs = frame_stack.step(obs)
                    ep_reward += reward
                    episode_rewards.append(ep_reward)

eval_env.close()

97 mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    print("-" * 50)
    print(f"Evaluation of {args.model_path} on {args.env_id}")
    print(f"Mean Reward: {mean_reward:.2f} +/- {std_reward:.2f}")
    print(f"Episodes: {args.n_eval_episodes}")
    print("-" * 50)

107 if __name__ == "__main__":
    main()

```

A.9 Plot Results (plot_results.py)

```

"""
2 Plot learning curves from experiment logs.

Produces:
    1. figures/lunar_ppo_vs_dqn.pdf - LunarLander PPO vs DQN comparison
    2. figures/pong_dqn_scratch.pdf - Pong DQN (from scratch) reward
7     curve
    3. figures/pong_ppo_sb3.pdf      - Pong PPO (SB3) reward curve from
        eval log

Usage:
    python plot_results.py
"""

12 import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
17 import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from pathlib import Path

plt.rcParams.update({
22     "font.family": "serif",
    "font.size": 12,
    "axes.labelsize": 13,
    "axes.titlesize": 14,

```

```

27     "legend.fontsize": 11,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11,
    "figure.dpi": 150,
})

32 FIGURES = Path("figures")
FIGURES.mkdir(exist_ok=True)

def smooth(values, window=10):
37     """Moving average smoothing."""
    kernel = np.ones(window) / window
    return np.convolve(values, kernel, mode="valid")

42 def load_sb3_eval_log(path: str):
    """
    Load SB3 EvalCallback npz log.
    Returns (timesteps, mean_rewards, std_rewards).
    """
47     data = np.load(path)
    return data["timesteps"], data["results"].mean(axis=1), data["results"]
        .std(axis=1)

def load_dqn_scratch_log(path: str):
52     """
    Load our custom JSON reward log.
    Returns (steps, mean_rewards).
    """
    with open(path) as f:
57         records = json.load(f)
        steps = [r["step"] for r in records]
        rewards = [r["mean_reward_10"] for r in records]
        return np.array(steps), np.array(rewards)

62 # --- Figure 1: LunarLander PPO vs DQN
    -----
ppo_lunar_log = "logs/ppo_lunar/evaluations.npz"
dqn_lunar_log = "logs/dqn_lunar_sb3/evaluations.npz"

67 if Path(ppo_lunar_log).exists() and Path(dqn_lunar_log).exists():
    fig, ax = plt.subplots(figsize=(7, 4.5))

    for label, log_path, color in [
72         ("PPO (SB3)", ppo_lunar_log, "#2563EB"),
        ("DQN (SB3)", dqn_lunar_log, "#DC2626"),
    ]:
        ts, means, stds = load_sb3_eval_log(log_path)
        ax.plot(ts, means, label=label, color=color, linewidth=1.8)
        ax.fill_between(ts, means - stds, means + stds, alpha=0.15, color=
            color)

```

```

77     ax.axhline(200, color="gray", linestyle="--", linewidth=1.0, label="
        Solved threshold")
    ax.set_xlabel("Environment Steps")
    ax.set_ylabel("Mean Episodic Reward (10 episodes)")
    ax.set_title("LunarLander-v3: PPO vs DQN Learning Curves")
82     ax.legend()
    ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x/1
        e6:.1f}M"))
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    fig.savefig(FIGURES / "lunar_ppo_vs_dqn.pdf")
87     fig.savefig(FIGURES / "lunar_ppo_vs_dqn.png")
    plt.close(fig)
    print("Saved: figures/lunar_ppo_vs_dqn.pdf")
else:
    print(f"[SKIP] LunarLander logs not found - run train_lunarlander.py
        first.")
92
# --- Figure 2: Pong DQN (from scratch)
-----
dqn_pong_log = "logs/dqn_pong_lr0.0001_seed42/rewards.json"

if Path(dqn_pong_log).exists():
97     steps, rewards = load_dqn_scratch_log(dqn_pong_log)
    smoothed = smooth(rewards, window=5)

    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot(steps, rewards, color="#16A34A", alpha=0.3, linewidth=1.0,
        label="Raw mean(10)")
102     ax.plot(steps[len(steps) - len(smoothed):], smoothed, color="#16A34A",
        linewidth=2.0, label="Smoothed (window=5)")

    ax.set_xlabel("Environment Steps")
    ax.set_ylabel("Mean Episodic Reward (last 10 eps)")
107     ax.set_title("Pong: DQN (from scratch) Learning Curve")
    ax.legend()
    ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x/1
        e6:.1f}M"))
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
112     fig.savefig(FIGURES / "pong_dqn_scratch.pdf")
    fig.savefig(FIGURES / "pong_dqn_scratch.png")
    plt.close(fig)
    print("Saved: figures/pong_dqn_scratch.pdf")
else:
117     print(f"[SKIP] DQN-Pong log not found - run train_dqn_pong.py first.")

# --- Figure 3: Pong PPO (SB3)
-----
ppo_pong_log = "logs/ppo_pong_lr0.00025_seed42/evaluations.npz"

122 if Path(ppo_pong_log).exists():
    ts, means, stds = load_sb3_eval_log(ppo_pong_log)

```



```

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(ts, means, color="#7C3AED", linewidth=2.0, label="PPO (SB3)")
ax.fill_between(ts, means - stds, means + stds, alpha=0.15, color="#7
C3AED")

ax.set_xlabel("Environment Steps")
ax.set_ylabel("Mean Episodic Reward (5 eval episodes)")
ax.set_title("Pong: PPO (SB3) Learning Curve")
ax.legend()
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x/1
e6:.1f}M"))
ax.grid(True, alpha=0.3)
plt.tight_layout()
fig.savefig(FIGURES / "pong_ppo_sb3.pdf")
fig.savefig(FIGURES / "pong_ppo_sb3.png")
plt.close(fig)
print("Saved: figures/pong_ppo_sb3.pdf")
else:
    print(f"[SKIP] PPO-Pong eval log not found - run train_ppo_pong.py
first.")

print("\nDone. All available figures saved in figures/")

```